

Amazon Sales Demand Prediction

Step-by-step explanation of *amazon_sales.ipynb*

The Big Picture

This notebook builds a **binary classifier** that predicts whether an Amazon product has **high or low demand**, using product attributes such as rating, price and discount. It trains **9 different models**, compares them fairly, picks the best one (**Gradient Boosting**), evaluates it in detail and saves it to disk.

Workflow: load data → define target → pick features → split → preprocess → define models → train and cross-validate → rank → report → save.

Cells 1-2: Imports and Setup

Cell 1 imports the core tools:

- pandas and numpy for data handling.
- `train_test_split`, `cross_val_score`, `StratifiedKFold` for splitting and validation.
- `ColumnTransformer` and `Pipeline` to chain preprocessing and modeling.
- `SimpleImputer` fills missing values; `StandardScaler` and `OneHotEncoder` transform numeric and categorical data.
- Metrics: accuracy, precision, recall, F1, confusion matrix and ROC-AUC measure performance.

Cell 2 imports the 9 candidate algorithms plus `joblib` (for saving the model), and sets `RANDOM_STATE = 42` so every random operation is **reproducible**.

Step 1 (Cell 3): Load the Data

```
df = pd.read_csv("amazon_products_sales_data_cleaned.csv")
```

The dataset has **9,663 rows and 17 columns**, one row per product.

Step 2 (Cell 4): Create the Target Variable

The dataset has no "high/low demand" column, so the notebook creates one from `purchased_last_month`:

```
df = df.dropna(subset=["purchased_last_month"]).copy()
median_purchases = df["purchased_last_month"].median() # 500
df["target"] = (df["purchased_last_month"] > median_purchases).astype(int)
```

- **Drop rows with no sales figure.** A product can't be labeled if its sales are unknown. This leaves **7,874 rows**.
- **Compute the median:** 500 purchases per month.
- **Label each product:** 1 (High Demand) if purchases are *greater than* 500, else 0 (Low Demand).

Why 60/40 and not 50/50: a median splits data evenly only if values don't tie. Many products sit at exactly 500, and `>` puts those in class 0, giving **60.5% Low / 39.5% High**.

Cell 5: `df.describe()`

Exploratory step showing count, mean, std, min, quartiles and max for each numeric column. Things worth noticing:

- **Missing values:** product_rating has 9,641 non-null values, discounted_price has 9,289, etc. This is why imputation is needed later.
- **Heavy skew:** total_reviews has a mean of 5,879 but a median of 894 and a max of 865,598.
- **Most products aren't discounted:** the median discount_percentage is 0.

Step 3 (Cell 6): Select Features

Type	Features
Numeric (5)	product_rating, total_reviews, discounted_price, original_price, discount_percentage
Categorical (5)	is_best_seller, is_sponsored, has_coupon, buy_box_availability, product_category

X holds the 10 input columns and y is the target. purchased_last_month is deliberately **excluded** from X: including it would be **data leakage**, since the target was derived directly from it.

Step 4 (Cell 7): Train/Test Split

```
train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

- **80% train (6,299 rows), 20% test (1,575 rows).**
- stratify=y keeps the 60/40 class ratio the same in both sets, so the test set isn't accidentally easier or harder.
- The test set is held back to simulate **unseen data**.

Step 5 (Cell 8): Preprocessing Pipeline

Models need clean, numeric input, so each data type is processed differently.

Numeric pipeline

- SimpleImputer(strategy="median") fills missing numbers with the column median, which is robust to outliers such as huge review counts.
- StandardScaler() rescales each feature to mean 0 and standard deviation 1. This matters for distance- or gradient-based models (Logistic Regression, KNN, SVM) and does no harm to tree models.

Categorical pipeline

- SimpleImputer(strategy="constant", fill_value="Unknown") turns missing categories into an explicit "Unknown" category.
- OneHotEncoder(handle_unknown="ignore", sparse_output=False) converts each category into 0/1 columns. handle_unknown="ignore" prevents crashes if the test data contains a category unseen in training.

ColumnTransformer applies the numeric pipeline to numeric columns and the categorical pipeline to categorical columns, then joins the results.

Why a pipeline matters: the imputer and scaler learn their statistics (medians, means) **only from training data**. Inside cross-validation, they are fit only on the training folds, which prevents data leakage.

Step 6 (Cell 9): Candidate Models

Model	What it does	Key setting
Logistic Regression	Linear model that outputs a probability	max_iter=1000
Decision Tree	Learns if/else splitting rules	max_depth=10 limits overfitting
Random Forest	200 trees on random subsets, then voted	n_estimators=200, max_depth=10
Extra Trees	Like Random Forest with more randomized splits	same settings
Gradient Boosting	Builds trees sequentially, each fixing the previous one's errors	defaults
AdaBoost	Boosting that re-weights misclassified samples	defaults
KNN	Majority vote of the 15 nearest products	n_neighbors=15
Naive Bayes (Gaussian)	Probabilistic model assuming independent, normally distributed features	defaults
SVM (RBF)	Finds a curved boundary separating classes	probability=True, max_iter=2000 (capped for speed)

They are stored in a dictionary so the next step can loop through them.

Step 7 (Cell 10): Train and Evaluate Every Model

This is the core of the notebook. For each model:

- **Build a full pipeline:** preprocessor, then classifier.
- **5-fold stratified cross-validation (training data only), scored by F1.** The training set is split into 5 parts; the model trains on 4 and validates on 1, rotating 5 times. `shuffle=True` randomizes rows and `n_jobs=-1` uses all CPU cores. You get the **mean** (expected performance) and **std** (stability).
- **Fit on the whole training set, then predict on the test set.** `predict` gives class labels; `predict_proba(...)[:, 1]` gives the probability of High Demand.
- **Compute test metrics** and store them in a results row, with the fitted pipeline kept in `fitted_pipelines`.

Metrics used

Metric	Meaning
Accuracy	Share of all predictions that are correct
Precision	Of products predicted High Demand, the share that really are
Recall	Of truly High Demand products, the share the model found
F1	Harmonic mean of precision and recall - a balanced score
ROC-AUC	How well the model ranks High above Low across all thresholds (0.5 = random, 1.0 = perfect)

SVM ConvergenceWarning: the solver stopped at `max_iter=2000` before fully converging. It is harmless here but means SVM's results are slightly under-optimized.

Cell 11: Model Comparison Charts

Two horizontal bar charts sorted best to worst: one for **Test F1** and one for **Test ROC-AUC**, so the ranking can be seen at a glance.

Step 8 (Cell 12): Rank and Pick the Winner

Results are sorted by Test F1 and the top model becomes `best_pipe`.

Rank	Model	CV F1	Test F1	Test ROC-AUC
1	Gradient Boosting	0.730	0.713	0.880
2	Random Forest	0.727	0.704	0.884
3	Decision Tree	0.716	0.698	0.833
4	AdaBoost	0.703	0.689	0.846
5	SVM (RBF)	0.684	0.657	0.859
6	KNN	0.665	0.656	0.832
7	Logistic Regression	0.659	0.629	0.829
8	Naive Bayes	0.589	0.603	0.738
9	Extra Trees	0.520	0.509	0.850

Notable results

- **Extra Trees** has 93.5% precision but only 35% recall. It is very cautious and rarely predicts "High", so it misses most high-demand products, which drags F1 down.
- **Naive Bayes** is the opposite: 97% recall but 44% precision and about 50% accuracy. It predicts "High" for almost everything; its Gaussian assumption fits poorly with one-hot encoded columns.
- **Tree-based ensembles dominate**, probably because demand depends on non-linear interactions (e.g. rating combined with review count and category).

Step 9 (Cell 13): Detailed Report for the Best Model

Class	Precision	Recall	F1-score	Support
Low Demand	0.79	0.89	0.84	954
High Demand	0.80	0.65	0.71	621
Accuracy			0.79	1575

Confusion matrix

	Predicted Low	Predicted High
Actually Low	851 (true negatives)	103 (false positives)
Actually High	220 (false negatives)	401 (true positives)

Reading it

- The model gets **79.5%** of products right overall (1,252 of 1,575).
- It is better at spotting **Low Demand** (89% recall) than **High Demand** (65% recall).
- The biggest weakness is the **220 false negatives**: high-demand products labeled low. If finding winners matters most, lowering the decision threshold trades some precision for more recall.

Cells 14-15: Report Table and Visuals

- **Cell 14** converts the classification report into a DataFrame for a cleaner table. `display()` works in Jupyter/Colab but not in plain Python scripts.

- **Cell 15** draws a **confusion matrix heatmap** and a **bar chart of precision/recall/F1 per class**, making the weaker High Demand recall visible.

Step 10 (Cell 16): Save Outputs

```
joblib.dump(best_pipe, "best_model.joblib")
results_df.to_csv("model_comparison_results.csv", index=False)
```

- The **entire pipeline** (preprocessing + Gradient Boosting) is saved as one file, so it can later predict on raw data with no manual preprocessing.
- The comparison table is saved as a CSV.

To reuse the saved model:

```
model = joblib.load("best_model.joblib")
model.predict(new_products_df) # needs the same 10 feature columns
```

Cells 17-18: Final Table and Written Justification

Cell 17 displays the rounded comparison table. Cell 18 prints a written argument for Gradient Boosting: highest CV and test F1, CV and test scores close together (little overfitting), low variance across folds, and a balance of precision and recall.

Things to Be Aware Of

- **The Gradient Boosting vs. Random Forest gap is tiny.** The F1 difference is under 0.01, within the CV standard deviation (about 0.015). Random Forest actually has the better ROC-AUC, so calling Gradient Boosting clearly "best" overstates it; they are effectively tied.
- **The target is an arbitrary cutoff.** "High demand" means more than 500 purchases, a median-based choice rather than a business-defined one.
- **Default hyperparameters.** Gradient Boosting, AdaBoost and others use defaults with no tuning (e.g. GridSearchCV). Tuning could improve results.
- **Winner chosen on the test set.** Ranking by Test F1 means the test set influenced model selection. CV F1 also picks Gradient Boosting, so the conclusion holds, but a separate final validation set would be cleaner.
- **Cells 19-24 are empty** and can be deleted.